

Gestión de bases de datos policiales

Importación, limpieza, diagnóstico, uniones y primeras visualizaciones

Noam López Villanes

13 de mayo de 2026

Tabla de contenidos

1	Objetivo de la sesión	2
2	Preparación	3
3	Importación	3
3.1	CSV: denuncias policiales	3
3.2	CSV: personal policial	3
3.3	CSV: parque automotor de la PNP	4
3.4	Excel: infraestructura de comisarías	5
3.5	SPSS: ejemplo de importación	6
4	Primer diagnóstico	7
4.1	Cuántas filas y columnas tiene cada base	7
4.2	Estandarizar nombres de variables	8
5	Valores perdidos	9
5.1	Conteo de valores perdidos por variable	9
5.2	Visualizar valores perdidos	10
6	Duplicados	13
6.1	Filas totalmente duplicadas	13
6.2	Duplicados según llaves sustantivas	14
7	Codificación y caracteres extraños	15
8	Estandarización de ubigeo	16
8.1	Verificar ubicaciones sin ubigeo	17

9	Primeras transformaciones	18
9.1	Total de denuncias por año	18
9.2	Principales modalidades	20
9.3	Personal policial por departamento	21
9.4	Estado del parque automotor	23
10	Fusionar bases de datos	25
10.1	Preparar denuncias por distrito	25
10.2	Preparar parque automotor por distrito	26
10.3	Leer ubicación de comisarías 2024	27
10.4	Hacer el join	29
10.5	Revisar el resultado del join	30
10.6	Visualizar relación entre denuncias y recursos	32
11	Documentar y validar la base	34
11.1	Diccionario mínimo de datos	35
11.2	Reglas de calidad	37
11.3	Diagnóstico de fusiones	38
11.4	Reestructurar datos: largo y ancho	40
12	Exportar una base limpia	42
13	Ejercicios para seguir practicando	44
13.1	Ejercicio 1: Gráfico interactivo con plotly	44

1. Objetivo de la sesión

En esta sesión trabajaremos con bases reales de información policial. El objetivo no es hacer todavía un análisis estadístico complejo, sino aprender una ruta ordenada para gestionar bases de datos:

1. Importar archivos CSV, Excel y SPSS.
2. Revisar estructura, nombres de variables y tipos de datos.
3. Detectar valores perdidos, duplicados y problemas de codificación.
4. Estandarizar variables clave, especialmente **ubigeo**.
5. Crear indicadores simples mediante agrupaciones.
6. Fusionar bases de datos con `left_join()`.
7. Elaborar primeras visualizaciones exploratorias.

2. Preparación

Si ejecutas este documento por partes, corre primero los chunks de **Instalar paquetes**, **Activar paquetes** y **Ubicar archivos**. Los tres están ocultos en el redenzado final, pero esos bloques crean los paquetes, funciones y rutas que usan los demás chunks.

3. Importación

3.1. CSV: denuncias policiales

La base de denuncias policiales contiene registros agregados por período, lugar y modalidad.

```
cargar_datos_originales()

denuncias <- read_csv(
  # read_csv() importa archivos separados por comas.
  file.path(ruta_csv, "denuncias.csv"),
  locale = locale(encoding = "UTF-8"),
  show_col_types = FALSE
)

# glimpse() muestra columnas, tipos y algunos valores de ejemplo.
glimpse(denuncias)
```

Rows: 355,947

Columns: 8

```
$ ANIO          <dbl> 2018, 2018, 2018, 2018, 2018, 2018, 2018, 2018, 2018, 2~
$ MES           <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ DPTO_HECHO_NEW <chr> "AMAZONAS", "AMAZONAS", "AMAZONAS", "AMAZONAS", "AMAZON~
$ PROV_HECHO    <chr> "BAGUA", "BAGUA", "BAGUA", "BAGUA", "BAGUA", "BAGUA", "~
$ DIST_HECHO    <chr> "ARAMANGO", "ARAMANGO", "BAGUA", "BAGUA", "BAGUA", "BAG~
$ UBIGEO_HECHO  <dbl> 10202, 10202, 10201, 10201, 10201, 10201, 10201, 10203,~
$ P_MODALIDADES <chr> "Otros", "Violencia contra la mujer e integrantes", "Es~
$ cantidad      <dbl> 6, 1, 2, 25, 64, 7, 8, 4, 1, 1, 2, 1, 2, 1, 2, 4, 7, 14~
```

3.2. CSV: personal policial

```

cargar_datos_originales()

personal <- read_csv(
  # La codificación UTF-8 permite leer correctamente tildes y eñes.
  file.path(ruta_csv, "personal.csv"),
  locale = locale(encoding = "UTF-8"),
  show_col_types = FALSE
)

glimpse(personal)

```

```

Rows: 19,187
Columns: 19
$ AÑO                <dbl> 2026, 2026, 2026, 2026, 2026, 2026, 2026, 2026, ~
$ MES                <chr> "MARZO", "MARZO", "MARZO", "MARZO", "MARZO", "MARZO", "M~
$ DEPENDENCIAS_GENERALES <chr> "COMANDANCIA GENERAL", "COMANDANCIA GENERAL", "~
$ DEPENDENCIAS       <chr> "DIRECCION DE PRESTACIONES DE SALUD", "DIRECCIO~
$ UNIDADES           <chr> "DIRECCION DE SANIDAD POLICIAL", "DIRECCION DE ~
$ DEPARTAMENTO        <chr> "TUMBES", "SAN MARTÍN", "LIMA METROPOLITANA", "~
$ PROVINCIA          <chr> "TUMBES", "SAN MARTIN", "LIMA", "LIMA", "TRUJIL~
$ DISTRITO           <chr> "TUMBES", "TARAPOTO", "RIMAC", "RIMAC", "TRUJIL~
$ SEXO               <chr> "MASCULINO", "FEMENINO", "MASCULINO", "MASCULIN~
$ CATEGORIA_OFI_SUB   <chr> "OFICIAL", "SUB OFICIAL", "OFICIAL", "OFICIAL", ~
$ CATEGORIA_ARM_SERV  <chr> "SERVICIOS", "ARMAS", "ARMAS", "ARMAS", "ARMAS"~
$ FUNCION            <chr> "ADM", "ADM", "ADM", "OPE", "OPE", "OPE", "ADM"~
$ APT.PSICOSOMATICA   <chr> "APTO COD. 0", "APTO COD. 0", "APTO COD. 0", "A~
$ ESTUDIOS_ADICIONALES <chr> "SIN ESTUDIOS ADICIONALES", "SIN ESTUDIOS ADICI~
$ PROFESION          <chr> "SIN ESTUDIOS ADICIONALES", "SIN ESTUDIOS ADICI~
$ ORIGINARIOS        <chr> "CASTELLANO", "CASTELLANO", "CASTELLANO", "CAST~
$ EXTRANJERO         <chr> "MATERNO", "MATERNO", "INGLÉS", "MATERNO", "MAT~
$ CLASIFICACION       <chr> "OTRAS DEPENDENCIAS", "OTRAS DEPENDENCIAS", "OT~
$ CANTIDAD            <dbl> 2, 8, 1, 13, 176, 1, 1, 2, 1, 1, 1, 1, 2, 1, 3, ~

```

3.3. CSV: parque automotor de la PNP

```

cargar_datos_originales()

parque <- read_csv(
  # Importamos el parque automotor como tabla de trabajo.
  file.path(ruta_csv, "parque.csv"),

```

```

  locale = locale(encoding = "UTF-8"),
  show_col_types = FALSE
)

glimpse(parque)

```

Rows: 8,232

Columns: 13

```

$ AÑO          <dbl> 2025, 2025, 2025, 2025, 2025, 2025, 2025, 2025, 202~
$ MES          <chr> "JULIO", "JULIO", "JULIO", "JULIO", "JULIO", "JULIO~
$ UBIGEO       <dbl> 150101, 150109, 40126, 210201, 170103, 150115, 1101~
$ NOMBDEPTO    <chr> "LIMA METROPOLITANA", "LIMA METROPOLITANA", "AREQUI~
$ NOMBPROV     <chr> "LIMA", "LIMA", "AREQUIPA", "AZANGARO", "TAMBOPATA~
$ NOMBDIST     <chr> "LIMA", "CIENEGUILLA", "YANAHUARA", "AZANGARO", "LA~
$ TIPO_VEHICULO <chr> "CAMIONETA", "CAMIONETA", "MOTOCICLETA", "CAMIONETA~
$ TIPO_UNIDAD  <chr> "UNIDAD ADMINISTRATIVA", "UNIDAD ESPECIALIZADA", "U~
$ TIPO_VEHICULO_PRIO <chr> "CAMIONETA", "CAMIONETA", "MOTOCICLETA", "CAMIONETA~
$ TIPO_PROCEDENCIA <chr> "COMPRA DIRECTA", "COMPRA DIRECTA", "COMPRA DIRECTA~
$ ESTADO       <chr> "INOPERATIVO IRRECUPERABLE", "INOPERATIVO IRRECUPER~
$ AÑOFACTURACION <dbl> 2016, 2016, 2015, 2008, 2010, 2024, 2017, 2025, 201~
$ CANTIDAD     <dbl> 11, 1, 3, 1, 1, 2, 1, 1, 2, 1, 1, 1, 1, 3, 1, 1,~

```

3.4. Excel: infraestructura de comisarías

```

cargar_datos_originales()

infraestructura <- read_excel(
  # read_excel() permite indicar archivo y hoja.
  file.path(
    ruta_infraestructura,
    "infraestructura.xlsx"
  ),
  sheet = "BD"
)

glimpse(infraestructura)

```

Rows: 1,319

Columns: 15

```

$ `N°` <dbl> ~
$ `MACRO REGIÓN` <chr> ~
$ `REGIÓN POLICIAL` <chr> ~
$ `NOMBRE DE LA COMISARÍA` <chr> ~
$ DEPARTAMENTO <chr> ~
$ PROVINCIA <chr> ~
$ DISTRITO <chr> ~
$ `ELECTRICIDAD\r\nADECUADO` <chr> ~
$ `AGUA\r\nADECUADO` <chr> ~
$ `DESAGUE\r\nADECUADO` <chr> ~
$ `INDICADOR 01: COMISARÍAS BÁSICAS CON\r\nSERVICIOS BÁSICOS ADECUADOS` <chr> ~
$ `PARED\r\nADECUADO` <chr> ~
$ `PISO\r\nADECUADO` <chr> ~
$ `TECHO\r\nADECUADO` <chr> ~
$ `INDICADOR 02: COMISARÍAS BÁSICAS CON\r\nINFRAESTRUCTURA EN BUEN ESTADO` <chr> ~

```

3.5. SPSS: ejemplo de importación

No usaremos esta base para todo el ejercicio, pero sirve para mostrar cómo importar archivos .sav.

```

cargar_datos_originales()

comisarias_lima_sav <- read_sav(
  # read_sav() importa archivos de SPSS.
  file.path(ruta_spss, "comisarias_lima.sav")
)

glimpse(comisarias_lima_sav)

```

Rows: 113

Columns: 16

```

$ `N°` <dbl> 649, 3, 411, 19, 407, 333, 11~
$ Macroregion <chr> "LIMA", "LIMA", "LIMA", "LIMA~
$ Regpol <chr> "LIMA", "LIMA", "LIMA", "LIMA~
$ Comisaria <chr> "CPNP SAN ISIDRO", "CPNP MAGD~
$ Departamento <chr> "LIMA METROPOLITANA", "LIMA M~
$ Provincia <chr> "LIMA", "LIMA", "LIMA", "LIMA~
$ Distrito <chr> "SAN ISIDRO", "MAGDALENA DEL ~
$ Electricidad_adeuada <chr> "SI", "SI", "SI", "SI", "SI",~
$ Agua_adeuada <chr> "SI", "SI", "SI", "SI", "SI",~

```

\$ Desague_adecuado	<chr> "NO", "SI", "SI", "SI", "SI", ~
\$ Indicador_de_servicios_básicos_adecuados	<chr> "NO", "SI", "SI", "SI", "SI", ~
\$ Paredes_adecuadas	<chr> "NO", "NO", "NO", "SI", "SI", ~
\$ Piso_adecuado	<chr> "NO", "SI", "NO", "SI", "NO", ~
\$ Techo_adecuado	<chr> "NO", "SI", "NO", "SI", "SI", ~
\$ Indicador_de_infraestructurada_adecuada	<chr> "NO", "NO", "NO", "SI", "NO", ~
\$ Cantidad_de_efectivos	<dbl> 9, 12, 15, 27, 29, 30, 32, 33~

4. Primer diagnóstico

4.1. Cuántas filas y columnas tiene cada base

```
cargar_datos_originales()

resumen_bases <- tibble(
  # Creamos una tabla resumen con tamaño de cada base.
  base = c(
    "denuncias",
    "personal",
    "parque",
    "infraestructura",
    "comisarias_lima_sav"
  ),
  filas = c(
    nrow(denuncias),
    nrow(personal),
    nrow(parque),
    nrow(infraestructura),
    nrow(comisarias_lima_sav)
  ),
  columnas = c(
    ncol(denuncias),
    ncol(personal),
    ncol(parque),
    ncol(infraestructura),
    ncol(comisarias_lima_sav)
  )
)

resumen_bases
```

```
# A tibble: 5 x 3
  base      filas columnas
  <chr>      <int>   <int>
1 denuncias 355947      8
2 personal  19187      19
3 parque    8232      13
4 infraestructura 1319      15
5 comisarias_lima_sav 113      16
```

La base de denuncias es la de mayor tamaño y será nuestra fuente principal para practicar agregaciones, limpieza y fusiones. Las otras bases funcionan como insumos complementarios para cruzar recursos policiales e infraestructura.

4.2. Estandarizar nombres de variables

Trabajar con nombres en mayúsculas, espacios, tildes o saltos de línea puede ser incómodo. `janitor::clean_names()` ayuda a convertirlos en nombres más consistentes.

```
preparar_bases_limpias()

# Revisamos cómo quedaron los nombres después de clean_names().
names(denuncias_limpia)
```

```
[1] "anio"          "mes"          "dpto_hecho_new" "prov_hecho"
[5] "dist_hecho"    "ubigeo_hecho" "p_modalidades"  "cantidad"
```

```
names(personal_limpia)
```

```
[1] "ano"          "mes"          "dependencias_generales"
[4] "dependencias" "unidades"     "departamento"
[7] "provincia"    "distrito"     "sexo"
[10] "categoria_ofi_sub" "categoria_arm_serv" "funcion"
[13] "apt_psicosomatica" "estudios_adicionales" "profesion"
[16] "originarios"    "extranjero"    "clasificacion"
[19] "cantidad"
```

```
names(parque_limpia)
```



```

[1] "ano"           "mes"           "ubigeo"
[4] "nombdepto"     "nombprov"       "nombdist"
[7] "tipo_vehiculo" "tipo_unidad"   "tipo_vehiculo_prio"
[10] "tipo_procedencia" "estado"        "anofabricacion"
[13] "cantidad"

```

```
names(infraestructura_limpia)
```

```

[1] "n"
[2] "macro_region"
[3] "region_policial"
[4] "nombre_de_la_comisaria"
[5] "departamento"
[6] "provincia"
[7] "distrito"
[8] "electricidad_adequado"
[9] "agua_adequado"
[10] "desague_adequado"
[11] "indicador_01_comisarias_basicas_con_servicios_basicos_adequados"
[12] "pared_adequado"
[13] "piso_adequado"
[14] "techo_adequado"
[15] "indicador_02_comisarias_basicas_con_infraestructura_en_buen_estado"

```

Después de `clean_names()`, los nombres quedan en minúsculas y sin espacios, lo que reduce errores al escribir código y facilita el uso de `select()`, `mutate()` y `group_by()`.

5. Valores perdidos

5.1. Conteo de valores perdidos por variable

```

preparar_bases_limpias()

faltantes_denuncias <- denuncias_limpia |>
  # Contamos cuántos NA tiene cada columna.
  summarise(across(everything(), ~ sum(is.na(.x)))) |>
  # Pasamos de muchas columnas a dos: variable y número de faltantes.
  pivot_longer(
    cols = everything(),

```

```

    names_to = "variable",
    values_to = "n_faltantes"
  ) |>
  mutate(pct_faltantes = n_faltantes / nrow(denuncias_limpia)) |>
  arrange(desc(n_faltantes))

faltantes_denuncias

```

```

# A tibble: 8 x 3
  variable      n_faltantes pct_faltantes
  <chr>          <int>         <dbl>
1 anio              0             0
2 mes               0             0
3 dpto_hecho_new    0             0
4 prov_hecho        0             0
5 dist_hecho        0             0
6 ubigeo_hecho      0             0
7 p_modalidades     0             0
8 cantidad          0             0

```

En esta base no solo interesa saber si hay valores perdidos, sino también en qué variables se concentran. Si los faltantes aparecen en una llave, como `ubigeo`, afectan directamente las fusiones posteriores.

5.2. Visualizar valores perdidos

```

if (sum(faltantes_denuncias$n_faltantes) == 0) {
  ggplot() +
    annotate(
      "text",
      x = 0.5,
      y = 0.5,
      label = "No se observan valores perdidos",
      size = 5
    ) +
    labs(title = "Valores perdidos en denuncias policiales") +
    theme_void(base_size = 11) +
    theme(
      plot.title = element_text(

```

```

      size = 13,
      hjust = 0.5,
      margin = margin(b = 10)
    ),
    plot.margin = margin(t = 16, r = 24, b = 16, l = 24)
  )
} else {
  limite_faltantes_denuncias <- max(faltantes_denuncias$pct_faltantes, 0.01)

  faltantes_denuncias |>
    filter(n_faltantes > 0) |>
    mutate(variable = fct_reorder(variable, pct_faltantes)) |>
    ggplot(aes(x = pct_faltantes, y = variable)) +
    geom_col(fill = "#8a817c") +
    scale_x_continuous(
      labels = percent_format(accuracy = 0.1),
      limits = c(0, limite_faltantes_denuncias),
      expand = expansion(mult = c(0, 0.05))
    ) +
    scale_y_discrete(labels = label_wrap(28)) +
    labs(
      title = "Valores perdidos en denuncias policiales",
      x = "% de filas con valor perdido",
      y = NULL
    ) +
    tema_sesion()
}

```

Valores perdidos en denuncias policiales

No se observan valores perdidos

El gráfico resume si hay campos incompletos. Cuando no aparecen faltantes, igualmente conviene mostrarlo: también es un hallazgo de calidad de datos.

```

preparar_bases_limpias()

faltantes_personal <- personal_limpia |>
  # Repetimos el diagnóstico para la base de personal policial.
  summarise(across(everything(), ~ sum(is.na(.x)))) |>
  pivot_longer(
    cols = everything(),
    names_to = "variable",
    values_to = "n_faltantes"
  ) |>
  mutate(pct_faltantes = n_faltantes / nrow(personal_limpia)) |>
  arrange(desc(n_faltantes))

if (sum(faltantes_personal$n_faltantes) == 0) {
  ggplot() +
    annotate(
      "text",
      x = 0.5,
      y = 0.5,
      label = "No se observan valores perdidos",
      size = 5
    ) +
    labs(title = "Valores perdidos en personal policial") +
    theme_void(base_size = 11) +
    theme(
      plot.title = element_text(
        size = 13,
        hjust = 0.5,
        margin = margin(b = 10)
      ),
      plot.margin = margin(t = 16, r = 24, b = 16, l = 24)
    )
} else {
  limite_faltantes_personal <- max(faltantes_personal$pct_faltantes, 0.01)

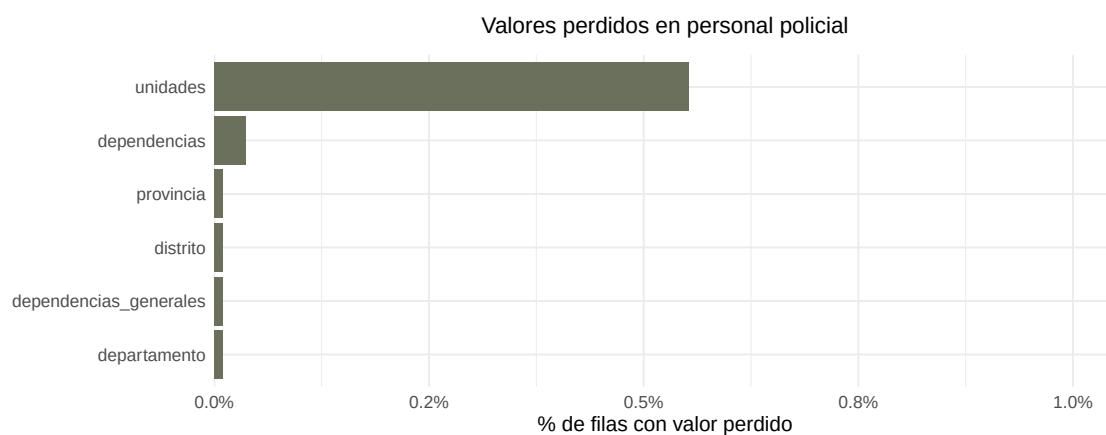
  faltantes_personal |>
    filter(n_faltantes > 0) |>
    mutate(variable = fct_reorder(variable, pct_faltantes)) |>
    ggplot(aes(x = pct_faltantes, y = variable)) +
    geom_col(fill = "#6b705c") +
    scale_x_continuous(
      labels = percent_format(accuracy = 0.1),

```

```

    limits = c(0, limite_faltantes_personal),
    expand = expansion(mult = c(0, 0.05))
  ) +
  scale_y_discrete(labels = label_wrap(28)) +
  labs(
    title = "Valores perdidos en personal policial",
    x = "% de filas con valor perdido",
    y = NULL
  ) +
  tema_sesion()
}

```



En personal policial sí aparecen faltantes en algunas variables administrativas. Esto no impide usar la base, pero alerta sobre campos que podrían requerir revisión antes de análisis más finos.

6. Duplicados

Un duplicado puede ser una fila totalmente repetida o una repetición según variables clave. Siempre debemos definir qué significa “duplicado” según la base.

6.1. Filas totalmente duplicadas

```
preparar_bases_limpias()

tibble(
  # Comparamos filas totales con filas únicas.
  base = c("denuncias", "personal", "parque", "infraestructura"),
  filas = c(
    nrow(denuncias_limpia),
    nrow(personal_limpia),
    nrow(parque_limpia),
    nrow(infraestructura_limpia)
  ),
  filas_unicas = c(
    nrow(distinct(denuncias_limpia)),
    nrow(distinct(personal_limpia)),
    nrow(distinct(parque_limpia)),
    nrow(distinct(infraestructura_limpia))
  )
) |>
  mutate(duplicados_exactos = filas - filas_unicas)
```

```
# A tibble: 4 x 4
  base          filas filas_unicas duplicados_exactos
<chr>         <int>      <int>          <int>
1 denuncias    355947    355947            0
2 personal     19187     19187            0
3 parque       8232      8232            0
4 infraestructura 1319      1319            0
```

Comparar filas totales y filas únicas permite detectar duplicados exactos. Si aparecen, el siguiente paso no es borrar automáticamente, sino verificar si el duplicado representa un error o una repetición válida de la fuente.

6.2. Duplicados según llaves sustantivas

En denuncias, una llave razonable puede ser año, mes, ubigeo y modalidad.

```
preparar_bases_limpias()

denuncias_duplicadas_clave <- denuncias_limpia |>
  # count() cuenta repeticiones por la llave que definimos.
```

```
count(anio, mes, ubigeo_hecho, p_modalidades, name = "n") |>
filter(n > 1) |>
arrange(desc(n))
```

```
denuncias_duplicadas_clave
```

```
# A tibble: 0 x 5
# i 5 variables: anio <dbl>, mes <dbl>, ubigeo_hecho <dbl>,
#   p_modalidades <chr>, n <int>
```

Aquí revisamos duplicados con una llave sustantiva. Esta mirada es más útil que buscar filas idénticas, porque pregunta si una misma combinación de año, mes, distrito y modalidad aparece más de una vez.

7. Codificación y caracteres extraños

Algunas bases públicas pueden venir con problemas de codificación. Un primer síntoma son caracteres como .

```
cargar_datos_originales()

trata_utf8 <- read_csv(
  # Primero intentamos leer con UTF-8.
  file.path(ruta_csv, "trata.csv"),
  locale = locale(encoding = "UTF-8"),
  show_col_types = FALSE
)

names(trata_utf8)
```

```
[1] "A\xd10"          "MES"
[3] "CCDD"            "lugar"
[5] "PROVINCIA"       "DISTRITO"
[7] "COMISAR\xcdA"    "categor\xeda"
[9] "Denuncias por comisi\xf3n" "Valor"
```

Si vemos caracteres como , probablemente la base fue guardada con otra codificación. Es una señal de alerta antes de limpiar nombres o analizar categorías de texto.

Si los nombres o valores aparecen con caracteres extraños, podemos probar otra codificación.

```
trata_latin1 <- read_csv(
  # Si aparecen caracteres extraños, probamos Latin1.
  file.path(ruta_csv, "trata.csv"),
  locale = locale(encoding = "Latin1"),
  show_col_types = FALSE
) |>
  clean_names()

names(trata_latin1)
```

```
[1] "ano"           "mes"           "ccdd"
[4] "lugar"         "provincia"     "distrito"
[7] "comisaria"     "categoria"     "denuncias_por_comision"
[10] "valor"
```

```
trata_latin1 |>
  select(1:8) |>
  head()
```

```
# A tibble: 6 x 8
   ano  mes  ccdd lugar  provincia distrito comisaria  categoria
<dbl> <dbl> <dbl> <chr>   <chr>      <chr>      <chr>      <chr>
1  2017    1    2 ÁNCASH SANTA    CHIMBOTE DIVICAJ -DEPINTRAP CAPTACIÓN
2  2017    1    2 ÁNCASH SANTA    CHIMBOTE DIVICAJ -DEPINTRAP CAPTACIÓN
3  2017    1    2 ÁNCASH SANTA    CHIMBOTE DIVICAJ -DEPINTRAP CAPTACIÓN
4  2017    1    2 ÁNCASH SANTA    CHIMBOTE DIVICAJ -DEPINTRAP CAPTACIÓN
5  2017    1    2 ÁNCASH SANTA    CHIMBOTE DIVICAJ -DEPINTRAP CAPTACIÓN
6  2017    1    2 ÁNCASH SANTA    CHIMBOTE DIVICAJ -DEPINTRAP CAPTACIÓN
```

Al leer con `Latin1`, los nombres y categorías pueden recuperarse mejor. Este tipo de prueba es importante cuando trabajamos con bases públicas descargadas de distintas plataformas.

8. Estandarización de ubigeo

El `ubigeo` debe tratarse como texto, no como número. Si se lee como número, se pueden perder ceros a la izquierda.


```
preparar_bases_limpias()

tibble(
  # Estos casos muestran por qué el ubigeo debe trabajarse como texto.
  ubigeo_original = c(10101, "150101", "70101.0", NA)
) |>
  mutate(ubigeo_estandar = formatear_ubigeo(ubigeo_original))
```

```
# A tibble: 4 x 2
  ubigeo_original ubigeo_estandar
  <chr>          <chr>
1 10101         010101
2 150101        150101
3 70101.0       070101
4 <NA>          <NA>
```

```
preparar_bases_trabajo()

denuncias_limpia |>
  count(ubigeo, sort = TRUE) |>
  head()
```

```
# A tibble: 6 x 2
  ubigeo      n
  <chr> <int>
1 070101   693
2 150101   689
3 150132   688
4 150135   684
5 150110   679
6 150103   675
```

El ubigeo se transforma a texto para conservar ceros a la izquierda. Esta decisión parece pequeña, pero es central: si el código se trata como número, varias fusiones pueden fallar silenciosamente.

8.1. Verificar ubicaciones sin ubigeo

```
preparar_bases_trabajo()

denuncias_limpia |>
  # Buscamos registros sin ubigeo válido.
  filter(is.na(ubigeo) | ubigeo == "000000") |>
  count(dpto_hecho_new, prov_hecho, dist_hecho, sort = TRUE)
```

```
# A tibble: 0 x 4
# i 4 variables: dpto_hecho_new <chr>, prov_hecho <chr>, dist_hecho <chr>,
#   n <int>
```

Si esta tabla devuelve registros, habría que revisar esos casos antes de fusionar. Un distrito sin ubigeo puede quedar fuera de cruces con parque automotor, comisarías u otras bases territoriales.

9. Primeras transformaciones

9.1. Total de denuncias por año

```
preparar_bases_trabajo()

denuncias_anio <- denuncias_limpia |>
  # Agrupamos por año y sumamos el número de denuncias.
  group_by(anio) |>
  summarise(
    denuncias = sum(cantidad, na.rm = TRUE),
    .groups = "drop"
  )

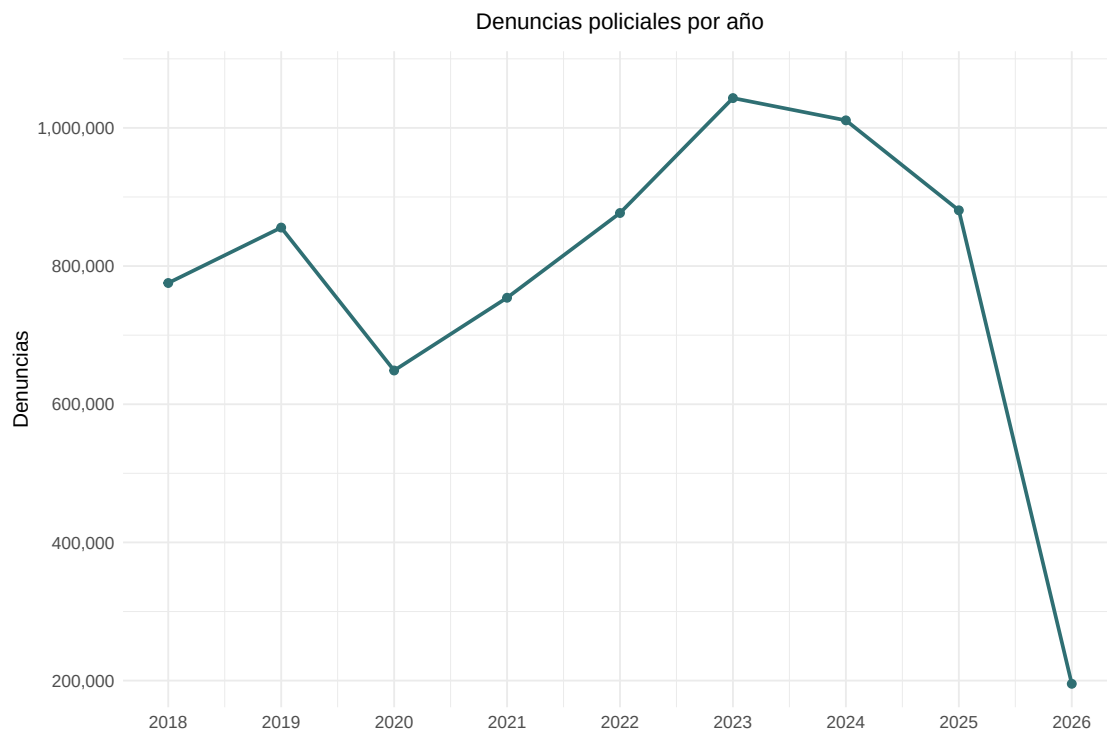
denuncias_anio
```

```
# A tibble: 9 x 2
  anio denuncias
  <int>     <dbl>
1  2018    775473
2  2019    855724
3  2020    648835
4  2021    754151
```

5	2022	876803
6	2023	1043136
7	2024	1010920
8	2025	880714
9	2026	195342

Esta agregación permite observar la serie anual de denuncias. El año 2026 debe interpretarse con cuidado porque la base llega solo hasta marzo, no al año completo.

```
ggplot(denuncias_anio, aes(x = anio, y = denuncias)) +
  geom_line(linewidth = 1, color = "#2f6f73") +
  geom_point(size = 2, color = "#2f6f73") +
  scale_x_continuous(breaks = denuncias_anio$anio) +
  scale_y_continuous(labels = comma, expand = expansion(mult = c(0.04, 0.08))) +
  labs(
    title = "Denuncias policiales por año",
    x = NULL,
    y = "Denuncias"
  ) +
  tema_sesion()
```



La caída de 2026 no debe leerse como reducción anual del delito: responde a que el año está incompleto. Este es un buen ejemplo de por qué siempre se revisa la fecha de corte.

9.2. Principales modalidades

```
preparar_bases_limpias()

modalidades_top <- denuncias_limpia |>
  # Agrupamos por modalidad y conservamos las principales categorías.
  group_by(p_modalidades) |>
  summarise(
    denuncias = sum(cantidad, na.rm = TRUE),
    .groups = "drop"
  ) |>
  slice_max(denuncias, n = 12) |>
  mutate(p_modalidades = fct_reorder(p_modalidades, denuncias))

modalidades_top
```

```
# A tibble: 7 x 2
  p_modalidades      denuncias
  <fct>            <dbl>
1 Otros            2688144
2 Violencia contra la mujer e integrantes  1864277
3 Hurto            1408214
4 Robo             753427
5 Estafa           205270
6 Extorsión        106729
7 Homicidio        15037
```

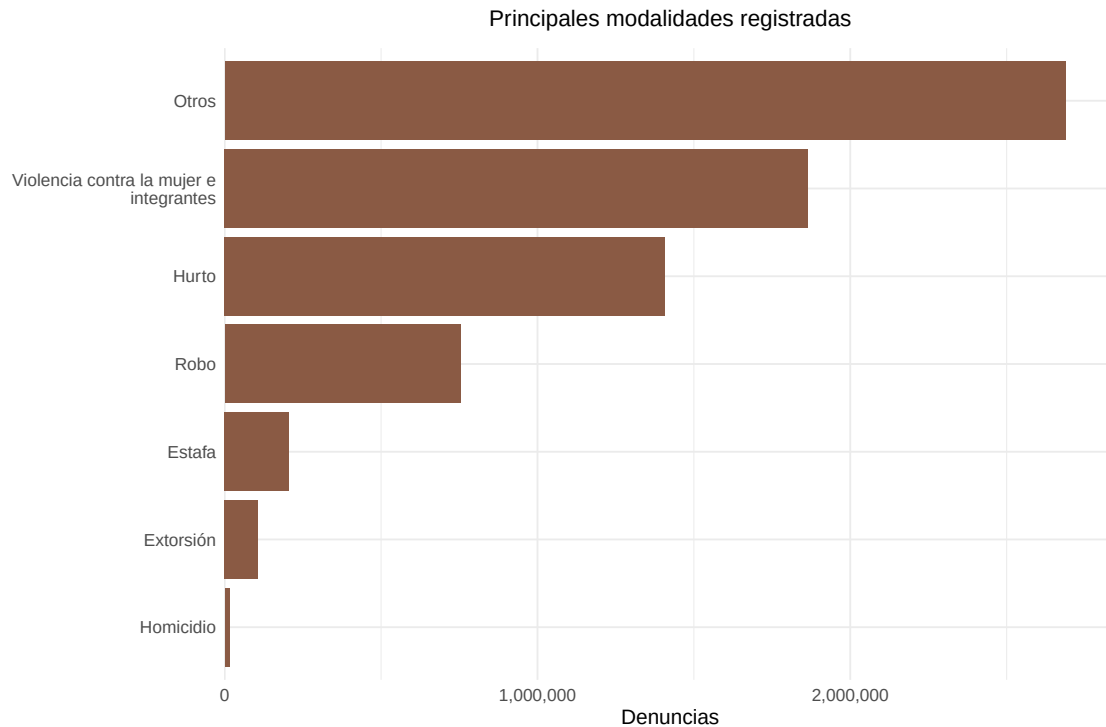
Las modalidades muestran que no todos los registros tienen el mismo peso analítico. Algunas categorías amplias, como “Otros”, pueden necesitar desagregación o discusión metodológica.

```
ggplot(modalidades_top, aes(x = denuncias, y = p_modalidades)) +
  geom_col(fill = "#8a5a44") +
  scale_x_continuous(labels = comma, expand = expansion(mult = c(0, 0.06))) +
  scale_y_discrete(labels = label_wrap(32)) +
  labs(
    title = "Principales modalidades registradas",
    x = "Denuncias",
```

```

y = NULL
) +
tema_sesion()

```



El gráfico ayuda a priorizar categorías para análisis posteriores. También muestra cuándo una categoría residual concentra demasiados casos y puede ocultar heterogeneidad.

9.3. Personal policial por departamento

```

preparar_bases_limpias()

personal_departamento <- personal_limpia |>
# Sumamos efectivos por departamento.
group_by(departamento) |>
summarise(
  efectivos = sum(cantidad, na.rm = TRUE),
  .groups = "drop"
)

```

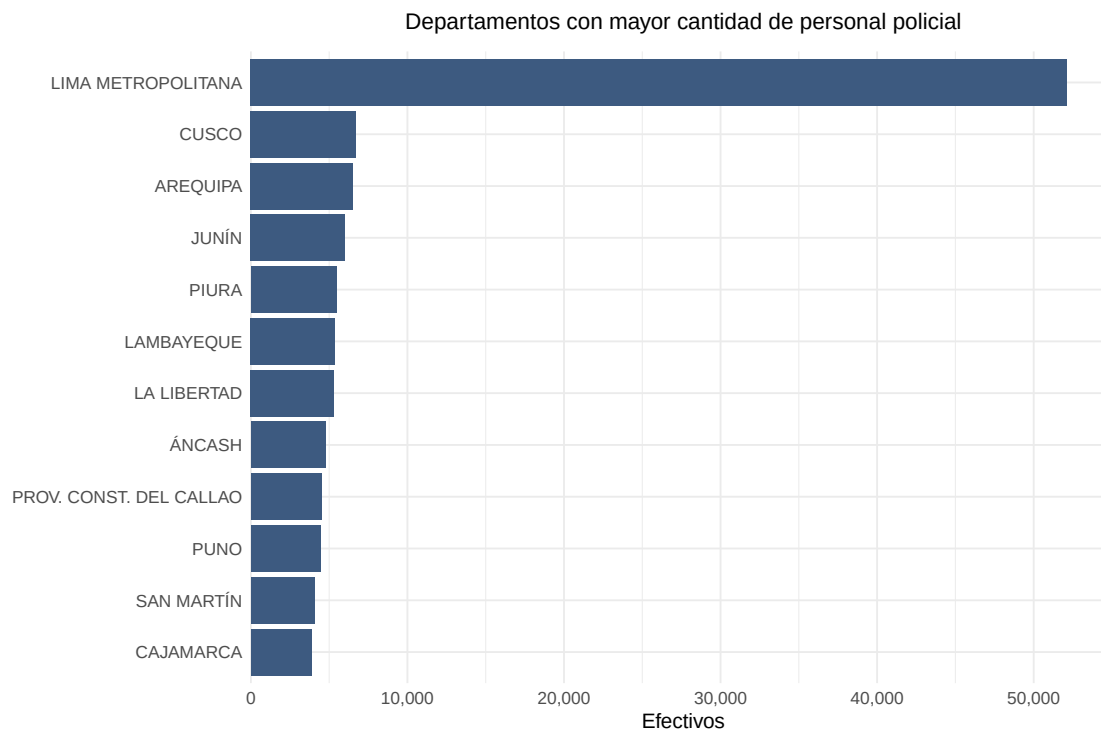
```
) |>
  arrange(desc(efectivos))

personal_departamento
```

```
# A tibble: 27 x 2
  departamento      efectivos
  <chr>            <dbl>
1 LIMA METROPOLITANA 52127
2 CUSCO              6715
3 AREQUIPA          6522
4 JUNÍN             6011
5 PIURA            5484
6 LAMBAYEQUE        5373
7 LA LIBERTAD       5309
8 ÁNCASH            4778
9 PROV. CONST. DEL CALLAO 4528
10 PUNO             4430
# i 17 more rows
```

Esta tabla resume la distribución territorial del personal. Para interpretarla mejor, luego podría cruzarse con población, denuncias o número de comisarías.

```
personal_departamento |>
  slice_max(efectivos, n = 12) |>
  mutate(departamento = fct_reorder(departamento, efectivos)) |>
  ggplot(aes(x = efectivos, y = departamento)) +
  geom_col(fill = "#3d5a80") +
  scale_x_continuous(labels = comma, expand = expansion(mult = c(0, 0.06))) +
  scale_y_discrete(labels = label_wrap(24)) +
  labs(
    title = "Departamentos con mayor cantidad de personal policial",
    x = "Efectivos",
    y = NULL
  ) +
  tema_sesion()
```



Lima Metropolitana concentra una parte importante del personal policial. Esto no necesariamente implica mayor cobertura relativa: para eso necesitaríamos tasas o indicadores por población.

9.4. Estado del parque automotor

```
preparar_bases_trabajo()

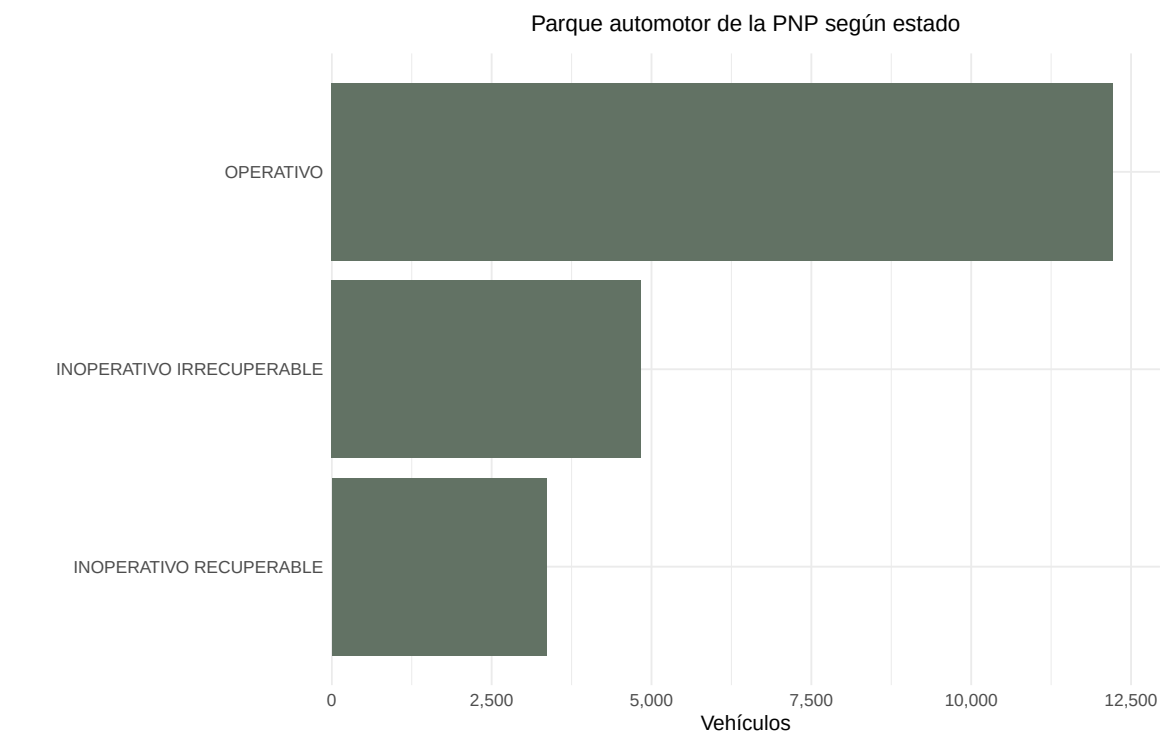
parque_estado <- parque_limpia |>
  # Sumamos vehículos según su estado operativo.
  group_by(estado) |>
  summarise(
    vehiculos = sum(cantidad, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(desc(vehiculos))

parque_estado
```

```
# A tibble: 3 x 2
  estado      vehiculos
  <chr>      <dbl>
1 OPERATIVO      12219
2 INOPERATIVO IRRECUPERABLE 4835
3 INOPERATIVO RECUPERABLE 3354
```

El estado del parque automotor permite distinguir recursos disponibles de recursos registrados. Para gestión pública, la diferencia entre operativo e inoperativo es sustantiva.

```
ggplot(parque_estado, aes(x = fct_reorder(estado, vehiculos), y = vehiculos)) +
  geom_col(fill = "#627264") +
  coord_flip() +
  scale_x_discrete(labels = label_wrap(26)) +
  scale_y_continuous(labels = comma, expand = expansion(mult = c(0, 0.06))) +
  labs(
    title = "Parque automotor de la PNP según estado",
    x = NULL,
    y = "Vehículos"
  ) +
  tema_sesion()
```

Esta visualización permite ver rápidamente la composición del parque automotor. Si una proporción alta está inoperativa, el total de vehículos puede sobreestimar la capacidad real.

10. Fusionar bases de datos

Para unir bases necesitamos una llave común. En este ejercicio usaremos `ubigeo`.

10.1. Preparar denuncias por distrito

```
preparar_bases_trabajo()

denuncias_distrito <- denuncias_limpia |>
  # Nos quedamos con los años recientes para el ejercicio de fusión.
  filter(anio >= 2024) |>
  group_by(ubigeo, llave_distrito, dpto_hecho_new, prov_hecho, dist_hecho) |>
  summarise(
```

```

    denuncias_2024_2026 = sum(cantidad, na.rm = TRUE),
    .groups = "drop"
)

denuncias_distrito

```

```

# A tibble: 1,808 x 6
  ubigeo llave_distrito          dpto_hecho_new prov_hecho dist_hecho
  <chr>   <chr>                <chr>         <chr>      <chr>
1 010101 AMAZONAS|CHACHAPOYAS|CHACHAPOYAS AMAZONAS     CHACHAPOYAS CHACHAPOY~
2 010102 AMAZONAS|CHACHAPOYAS|ASUNCION    AMAZONAS     CHACHAPOYAS ASUNCION
3 010103 AMAZONAS|CHACHAPOYAS|BALSAS        AMAZONAS     CHACHAPOYAS BALSAS
4 010104 AMAZONAS|CHACHAPOYAS|CHETO        AMAZONAS     CHACHAPOYAS CHETO
5 010105 AMAZONAS|CHACHAPOYAS|CHILIQUN    AMAZONAS     CHACHAPOYAS CHILIQUN
6 010106 AMAZONAS|CHACHAPOYAS|CHUQUIBAMBA AMAZONAS     CHACHAPOYAS CHUQUIBAM~
7 010107 AMAZONAS|CHACHAPOYAS|GRANADA    AMAZONAS     CHACHAPOYAS GRANADA
8 010108 AMAZONAS|CHACHAPOYAS|HUANCAS     AMAZONAS     CHACHAPOYAS HUANCAS
9 010109 AMAZONAS|CHACHAPOYAS|LA JALCA    AMAZONAS     CHACHAPOYAS LA JALCA
10 010110 AMAZONAS|CHACHAPOYAS|LEIMEBAMBA AMAZONAS     CHACHAPOYAS LEIMEBAMBA
# i 1,798 more rows
# i 1 more variable: denuncias_2024_2026 <dbl>

```

Al agregar por distrito pasamos de una base de eventos agregados por modalidad y mes a una base territorial. Esta será la unidad que usaremos para cruzar información con recursos policiales.

10.2. Preparar parque automotor por distrito

```

preparar_bases_trabajo()

parque_distrito <- parque_limpia |>
  # Agregamos vehículos por distrito antes de unir.
  group_by(ubigeo, nombdepto, nombprov, nombdist) |>
  summarise(
    vehiculos_pnp = sum(cantidad, na.rm = TRUE),
    vehiculos_operativos = sum(cantidad[estado == "OPERATIVO"], na.rm = TRUE),
    .groups = "drop"
  )

parque_distrito

```

```
# A tibble: 1,013 x 6
  ubigeo nombdepto nombprov    nombdist vehiculos_pnp vehiculos_operativos
  <chr>   <chr>     <chr>    <chr>         <dbl>         <dbl>
1 010101 AMAZONAS  CHACHAPOYAS CHACHAPOYAS      65           30
2 010201 AMAZONAS  BAGUA       BAGUA           10            6
3 010202 AMAZONAS  BAGUA       ARAMANGO        2            1
4 010203 AMAZONAS  BAGUA       COPALLIN        2            1
5 010205 AMAZONAS  BAGUA       IMAZA           1            1
6 010301 AMAZONAS  BONGARA     JUMBILLA        2            1
7 010306 AMAZONAS  BONGARA     FLORIDA         4            2
8 010307 AMAZONAS  BONGARA     JAZAN          12            9
9 010311 AMAZONAS  BONGARA     VALERA          1            0
10 010401 AMAZONAS  CONDORCANQUI NIEVA           3            3
# i 1,003 more rows
```

El parque automotor se resume por distrito para que tenga la misma unidad de análisis que las denuncias. Antes de fusionar, cada base debe estar al mismo nivel de agregación.

10.3. Leer ubicación de comisarías 2024

La hoja CPNP_GEO tiene información georreferenciada de comisarías. En esta versión, el campo `cod_inei` no corresponde al ubigeo completo de seis dígitos; por eso construiremos una llave textual con departamento, provincia y distrito.

```
preparar_bases_trabajo()

comisarias_geo <- read_csv(
  # Leemos la base ligera con ubicación de comisarías.
  file.path(ruta_comisarias, "comisarias_geo.csv"),
  show_col_types = FALSE
) |>
  mutate(
    llave_distrito = paste(
      normalizar_texto(departamen),
      normalizar_texto(provincia),
      normalizar_texto(distrito),
      sep = "|"
    )
  )

glimpse(comisarias_geo)
```

Rows: 1,326

Columns: 25

```
$ n <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
$ clas_muni <chr> "A", "A", "B", "G", "G", "B", "D", "D", "D", "A", "B", ~
$ cod_inei <chr> "0944", "0945", "0272", "0928", "1102", "1114", "0601", ~
$ cod_cnpn <chr> "4525", "4532", "1529", "4550", "5908", "5920", "3105", ~
$ comisaria <chr> "CPNP JULIACA", "CPNP SANTA BARBARA", "CPNP SANTA MARIA~
$ tipo_comi <chr> "A", "B", "A", "E", "C", "B", "B", "B", "C", "B", "D", ~
$ cod_uni <dbl> 219303, 219307, 222078, 219335, 217489, 217493, 217909, ~
$ zona_utm <chr> "UTM / 19 SUR / WGS84", "UTM / 19 SUR / WGS84", "UTM / ~
$ categoria <chr> "PUNO", "PUNO", "AMAZONAS", "PUNO", "HUANUCO", "HUANUCO~
$ resolucion <chr> "APROBADO CON RESOLUCION DE COMANDANCIA GENERAL DE LA P~
$ id_dpto <chr> "21", "21", "01", "21", "10", "10", "12", "12", "12", "~
$ departamen <chr> "PUNO", "PUNO", "AMAZONAS", "PUNO", "HUANUCO", "HUANUCO~
$ id_prov <chr> "2111", "2111", "0104", "2106", "1001", "1008", "1203", ~
$ provincia <chr> "SAN ROMAN", "SAN ROMAN", "CONDORCANQUI", "HUANCANE", "~
$ id_dist <chr> "211101", "211101", "010401", "210607", "100103", "1008~
$ distrito <chr> "JULIACA", "JULIACA", "NIEVA", "TARACO", "CHINCHAO", "P~
$ cod_divpol <chr> "18R1A", "18R1A", "19R1A", "18R1A", "08R1B", "08R1B", "~
$ divpol_div <chr> "DIVOPUS JULIACA", "DIVOPUS JULIACA", "DIVOPUS AMAZONAS~
$ cod_macror <chr> "18", "18", "19", "18", "08", "08", "10", "10", "10", "~
$ macroregpo <chr> "PUNO", "PUNO", "SAN MARTIN", "PUNO", "HUANUCO", "HUANU~
$ cod_regpol <chr> "18R1", "18R1", "19R1", "18R1", "08R1", "08R1", "10R2", ~
$ regionpol <chr> "PUNO", "PUNO", "AMAZONAS", "PUNO", "HUANUCO", "HUANUCO~
$ latitud <dbl> -15.492585, -15.504741, -4.592240, -15.297966, -
9.80165~
$ longitud <dbl> -70.12805, -70.12801, -77.86418, -69.97882, -76.07137, ~
$ llave_distrito <chr> "PUNO|SAN ROMAN|JULIACA", "PUNO|SAN ROMAN|JULIACA", "AM~
```

Esta base aporta ubicación de comisarías, pero su código `cod_inei` no es directamente comparable con el `ubigeo`. Por eso usamos una llave textual normalizada como solución práctica para la sesión.

```
if (!exists("comisarias_geo")) {
  preparar_fusiones()
}

comisarias_distrito <- comisarias_geo |>
# Contamos comisarías únicas por distrito normalizado.
group_by(llave_distrito) |>
summarise(
  comisarias = n_distinct(cod_cnpn),
```

```

    .groups = "drop"
  )

comisarias_distrito

```

```

# A tibble: 1,059 x 2
  llave_distrito      comisarias
  <chr>              <int>
1 Amazonas|BAGUA|ARAMANGO      2
2 Amazonas|BAGUA|BAGUA         1
3 Amazonas|BAGUA|COPALLIN      1
4 Amazonas|BAGUA|IMAZA         2
5 Amazonas|BAGUA|LA PECA        1
6 Amazonas|BONGARA|CHURUJA      1
7 Amazonas|BONGARA|FLORIDA      1
8 Amazonas|BONGARA|JAZAN        1
9 Amazonas|BONGARA|JUMBILLA     1
10 Amazonas|BONGARA|VALERA       1
# i 1,049 more rows

```

Contar comisarias por distrito permite pasar de una base de establecimientos a un indicador territorial comparable con denuncias y vehículos.

10.4. Hacer el join

```

preparar_fusiones()

base_distrital <- denuncias_distrito |>
  # left_join() conserva todos los distritos de denuncias.
  left_join(
    parque_distrito |> select(ubigeo, vehiculos_pnp, vehiculos_operativos),
    by = "ubigeo"
  ) |>
  # Usamos llave textual porque el código INEI de comisarias no es ubigeo completo.
  left_join(comisarias_distrito, by = "llave_distrito") |>
  mutate(
    # Si no hubo cruce, asumimos cero recursos registrados en esta base.
    vehiculos_pnp = replace_na(vehiculos_pnp, 0),
    vehiculos_operativos = replace_na(vehiculos_operativos, 0),

```

```

    comisarias = replace_na(comisarias, 0)
  )

base_distrital

```

```

# A tibble: 1,808 x 9
  ubigeo llave_distrito          dpto_hecho_new prov_hecho dist_hecho
  <chr>   <chr>                <chr>         <chr>      <chr>
1 010101 AMAZONAS|CHACHAPOYAS|CHACHAPOYAS AMAZONAS    CHACHAPOYAS CHACHAPOY~
2 010102 AMAZONAS|CHACHAPOYAS|ASUNCION   AMAZONAS    CHACHAPOYAS ASUNCION
3 010103 AMAZONAS|CHACHAPOYAS|BALSAS      AMAZONAS    CHACHAPOYAS BALSAS
4 010104 AMAZONAS|CHACHAPOYAS|CHETO        AMAZONAS    CHACHAPOYAS CHETO
5 010105 AMAZONAS|CHACHAPOYAS|CHILIQUN    AMAZONAS    CHACHAPOYAS CHILIQUN
6 010106 AMAZONAS|CHACHAPOYAS|CHUQUIBAMBA AMAZONAS    CHACHAPOYAS CHUQUIBAM~
7 010107 AMAZONAS|CHACHAPOYAS|GRANADA    AMAZONAS    CHACHAPOYAS GRANADA
8 010108 AMAZONAS|CHACHAPOYAS|HUANCAS    AMAZONAS    CHACHAPOYAS HUANCAS
9 010109 AMAZONAS|CHACHAPOYAS|LA JALCA   AMAZONAS    CHACHAPOYAS LA JALCA
10 010110 AMAZONAS|CHACHAPOYAS|LEIMEBAMBA AMAZONAS    CHACHAPOYAS LEIMEBAMBA
# i 1,798 more rows
# i 4 more variables: denuncias_2024_2026 <dbl>, vehiculos_pnp <dbl>,
#   vehiculos_operativos <dbl>, comisarias <int>

```

La base integrada conserva todos los distritos con denuncias recientes y agrega, cuando existe cruce, información de vehículos y comisarías. Los ceros deben interpretarse como “sin registro cruzado en estas bases”, no necesariamente como ausencia real.

10.5. Revisar el resultado del join

```

if (!exists("base_distrital")) {
  preparar_fusiones()
}

base_distrital |>
  summarise(
    distritos = n(),
    distritos_con_vehiculos = sum(vehiculos_pnp > 0),
    distritos_con_comisarias = sum(comisarias > 0),
    denuncias_total = sum(denuncias_2024_2026, na.rm = TRUE)
  )

```

```
# A tibble: 1 x 4
  distritos distritos_con_vehiculos distritos_con_comisarias denuncias_total
    <int>          <int>          <int>          <dbl>
1     1808          1009           960        2086976
```

Este resumen muestra la cobertura del cruce: cuántos distritos tienen información complementaria de vehículos o comisarías y cuántas denuncias acumula la base final.

```
if (!exists("base_distrital")) {
  preparar_fusiones()
}

base_distrital |>
  arrange(desc(denuncias_2024_2026)) |>
  select(
    dpto_hecho_new,
    prov_hecho,
    dist_hecho,
    denuncias_2024_2026,
    vehiculos_pnp,
    comisarias
  ) |>
  head(20)
```

```
# A tibble: 20 x 6
  dpto_hecho_new      prov_hecho dist_hecho denuncias_2024_2026 vehiculos_pnp
    <chr>          <chr>      <chr>          <dbl>          <dbl>
1 LIMA METROPOLITANA LIMA        LIMA            74349          2692
2 LIMA METROPOLITANA LIMA        SAN JUAN ~      73859           225
3 LAMBAYEQUE         CHICLAYO    CHICLAYO        56532           383
4 PROV. CONST. DEL CAL~ CALLAO      CALLAO          48771           547
5 LIMA METROPOLITANA LIMA        COMAS           42326           114
6 LA LIBERTAD        TRUJILLO    TRUJILLO        42233           370
7 LIMA METROPOLITANA LIMA        SAN MARTI~      38728            88
8 LIMA METROPOLITANA LIMA        ATE             37817           272
9 LIMA METROPOLITANA LIMA        VILLA MAR~     33031           114
10 LIMA METROPOLITANA LIMA        LOS OLIVOS      31903           142
11 LIMA METROPOLITANA LIMA        SANTIAGO ~     31594            86
12 LIMA METROPOLITANA LIMA        VILLA EL ~     28725            32
13 PIURA            PIURA      PIURA          27224           316
14 LIMA METROPOLITANA LIMA        SAN JUAN ~     25753            74
15 LIMA METROPOLITANA LIMA        INDEPENDEN~    25662            89
```

16	LIMA METROPOLITANA	LIMA	MIRAFLORES	23941	35
17	LIMA METROPOLITANA	LIMA	CHORRILLOS	23879	134
18	LIMA METROPOLITANA	LIMA	CARABAYLLO	23802	63
19	LIMA METROPOLITANA	LIMA	LA VICTOR~	22137	290
20	ICA	ICA	ICA	21556	309

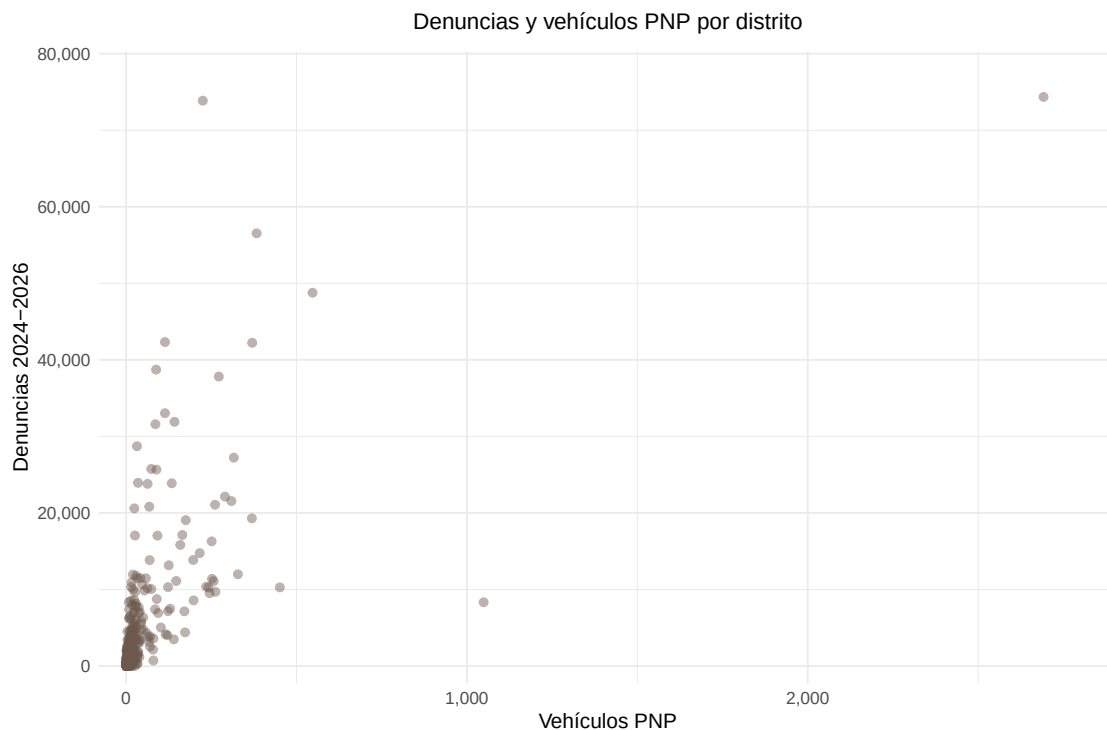
i 1 more variable: comisarias <int>

Revisar los distritos con más denuncias permite detectar casos influyentes y decidir si conviene analizarlos por separado, especialmente cuando hay grandes ciudades o distritos capitales.

10.6. Visualizar relación entre denuncias y recursos

```
if (!exists("base_distrital")) {
  preparar_fusiones()
}

base_distrital |>
  filter(vehiculos_pnp > 0) |>
  ggplot(aes(x = vehiculos_pnp, y = denuncias_2024_2026)) +
  geom_point(alpha = 0.45, color = "#6c584c", size = 2) +
  scale_x_continuous(labels = comma, expand = expansion(mult = c(0.03, 0.08))) +
  scale_y_continuous(labels = comma, expand = expansion(mult = c(0.03, 0.08))) +
  labs(
    title = "Denuncias y vehículos PNP por distrito",
    x = "Vehículos PNP",
    y = "Denuncias 2024-2026"
  ) +
  tema_sesion()
```

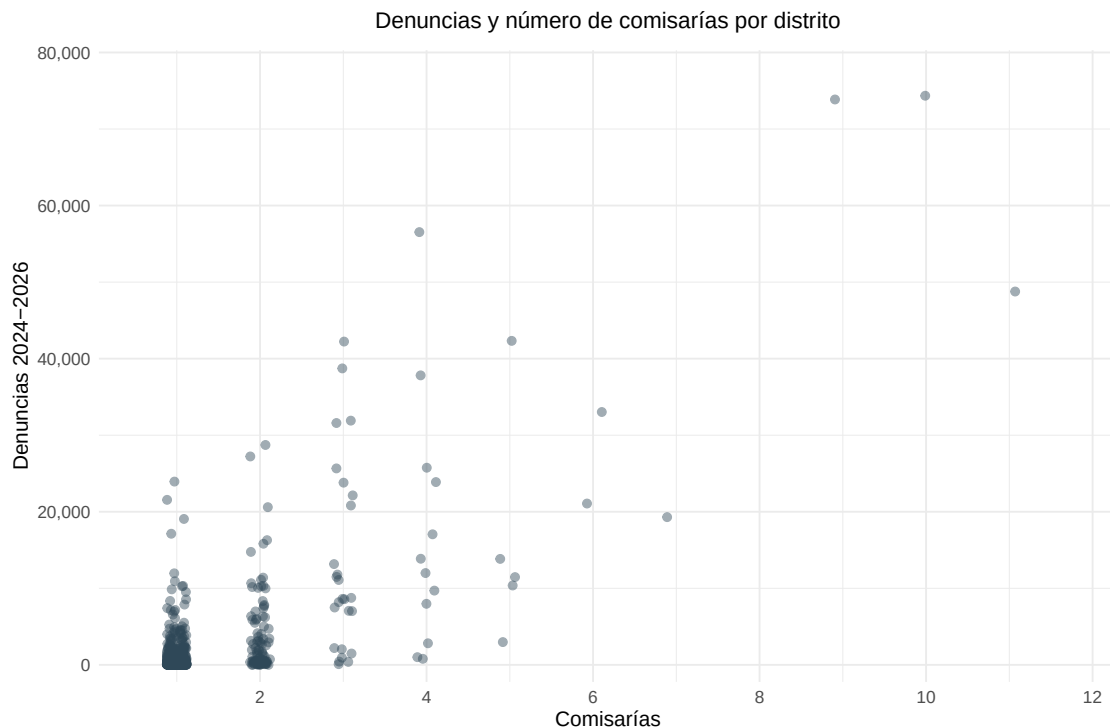



La relación entre denuncias y vehículos no debe interpretarse causalmente. En esta etapa solo exploramos si hay patrones, concentraciones o valores extremos que merezcan revisión.

```
if (!exists("base_distrital")) {
  preparar_fusiones()
}

base_distrital |>
  filter(comisarias > 0) |>
  ggplot(aes(x = comisarias, y = denuncias_2024_2026)) +
  geom_jitter(
    width = 0.12,
    height = 0,
    alpha = 0.45,
    color = "#2f4858",
    size = 2
  ) +
  scale_x_continuous(
    breaks = pretty_breaks(),
    expand = expansion(mult = c(0.08, 0.12))
  )
```

```
) +
scale_y_continuous(labels = comma, expand = expansion(mult = c(0.03, 0.08))) +
labs(
  title = "Denuncias y número de comisarías por distrito",
  x = "Comisarías",
  y = "Denuncias 2024-2026"
) +
tema_sesion()
```



Los distritos con más comisarías tienden a ser territorios urbanos o con mayor actividad policial registrada. Esta relación requiere contexto territorial antes de extraer conclusiones.

11. Documentar y validar la base

Antes de exportar una base limpia conviene dejar evidencia de cuatro cosas: de dónde vienen los datos, qué reglas de calidad aplicamos, qué tan bien cruzaron las bases y si necesitamos cambiar la forma de la tabla para analizarla mejor.

11.1. Diccionario mínimo de datos

Un diccionario no tiene que ser largo para ser útil. En una primera versión basta con identificar la fuente, la unidad de análisis y el papel de cada variable.

```
if (!exists("base_distrital")) {  
  preparar_fusiones()  
}  
  
diccionario_base_distrital <- tibble(  
  # Documentamos variables, tipos y significado.  
  variable = names(base_distrital),  
  tipo = map_chr(base_distrital, ~ class(.x)[1]),  
  descripcion = case_when(  
    variable == "ubigeo" ~ "Código de seis dígitos del distrito.",  
    variable ==  
      "llave_distrito" ~ "Llave textual normalizada: departamento, provincia y distrito.",  
    variable ==  
      "dpto_hecho_new" ~ "Departamento donde ocurrió el hecho registrado.",  
    variable == "prov_hecho" ~ "Provincia donde ocurrió el hecho registrado.",  
    variable == "dist_hecho" ~ "Distrito donde ocurrió el hecho registrado.",  
    variable ==  
      "denuncias_2024_2026" ~ "Total de denuncias policiales entre 2024 y marzo de 2026.",  
    variable ==  
      "vehiculos_pnp" ~ "Total de vehículos PNP registrados en el distrito.",  
    variable ==  
      "vehiculos_operativos" ~ "Total de vehículos PNP operativos en el distrito.",  
    variable ==  
      "comisarias" ~ "Número de comisarías identificadas en el distrito.",  
    TRUE ~ "Variable auxiliar o proveniente de la fuente original."  
  )  
)  
  
diccionario_base_distrital
```

```
# A tibble: 9 x 3  
  variable      tipo      descripcion  
  <chr>        <chr>    <chr>  
1 ubigeo      character Código de seis dígitos del distrito.  
2 llave_distrito character Llave textual normalizada: departamento, provi~  
3 dpto_hecho_new character Departamento donde ocurrió el hecho registrado.  
4 prov_hecho  character Provincia donde ocurrió el hecho registrado.
```

5	dist_hecho	character	Distrito donde ocurrió el hecho registrado.
6	denuncias_2024_2026	numeric	Total de denuncias policiales entre 2024 y mar~
7	vehiculos_pnp	numeric	Total de vehículos PNP registrados en el distr~
8	vehiculos_operativos	numeric	Total de vehículos PNP operativos en el distri~
9	comisarias	integer	Número de comisarías identificadas en el distr~

El diccionario hace explícito qué significa cada variable. Esto es especialmente importante cuando una base será compartida o reutilizada en otra sesión.

También es útil registrar una ficha breve de trazabilidad de las bases utilizadas.

```
trazabilidad <- tibble(
  # Esta ficha permite recordar fuente, unidad y uso de cada archivo.
  base = c(
    "Denuncias policiales",
    "Parque automotor PNP",
    "Comisarías georreferenciadas"
  ),
  archivo = c(
    "denuncias.csv",
    "parque.csv",
    "Información policial_BDs_seguridad_24.xlsx"
  ),
  unidad_original = c(
    "Año, mes, distrito y modalidad",
    "Distrito, tipo de vehículo, estado y procedencia",
    "Comisaría"
  ),
  uso_en_la_sesion = c(
    "Construir denuncias acumuladas por distrito",
    "Agregar vehículos por distrito",
    "Contar comisarías por distrito"
  )
)

trazabilidad
```

A tibble: 3 x 4

	base	archivo	unidad_original	uso_en_la_sesion
	<chr>	<chr>	<chr>	<chr>
1	Denuncias policiales	denuncias.csv	Año, mes, dist~	Construir denun~
2	Parque automotor PNP	parque.csv	Distrito, tipo~	Agregar vehícul~
3	Comisarías georreferenciadas	Información pol~	Comisaría	Contar comisari~

La trazabilidad permite reconstruir de dónde salió cada indicador. En análisis aplicado, esto ayuda a evitar confundir fuentes, fechas de corte o unidades de análisis.

11.2. Reglas de calidad

Las reglas de calidad son pruebas simples que nos avisan si algo no cuadra. No reemplazan el criterio sustantivo, pero ayudan a detectar errores temprano.

```
if (!exists("base_distrital")) {  
  preparar_fusiones()  
}  
  
validaciones <- tibble(  
  # Cada regla cuenta cuántos registros incumplen una condición básica.  
  regla = c(  
    "El ubigeo debe tener seis dígitos",  
    "Las denuncias no pueden ser negativas",  
    "Los vehículos no pueden ser negativos",  
    "Los vehículos operativos no pueden superar el total de vehículos",  
    "El número de comisarias no puede ser negativo"  
  ),  
  registros_con_problema = c(  
    sum(!str_detect(base_distrital$ubigeo, "^\\d{6}$"), na.rm = TRUE),  
    sum(base_distrital$denuncias_2024_2026 < 0, na.rm = TRUE),  
    sum(base_distrital$vehiculos_pnp < 0, na.rm = TRUE),  
    sum(  
      base_distrital$vehiculos_operativos > base_distrital$vehiculos_pnp,  
      na.rm = TRUE  
    ),  
    sum(base_distrital$comisarias < 0, na.rm = TRUE)  
  )  
)  
  
validaciones
```

A tibble: 5 x 2

regla <chr>	registros_con_problema <int>
1 El ubigeo debe tener seis dígitos	0
2 Las denuncias no pueden ser negativas	0
3 Los vehículos no pueden ser negativos	0
4 Los vehículos operativos no pueden superar el total de~	0

Si una regla devuelve cero problemas, no significa que la base sea perfecta, pero sí que pasó controles básicos de consistencia. Si devuelve casos, esos registros deben revisarse antes de exportar.

```
if (!exists("base_distrital")) {
  preparar_fusiones()
}

base_distrital |>
  filter(
    !str_detect(ubigeo, "^\\d{6}$") |
    denuncias_2024_2026 < 0 |
    vehiculos_pnp < 0 |
    vehiculos_operativos > vehiculos_pnp |
    comisarias < 0
  )
```

```
# A tibble: 0 x 9
#   i 9 variables: ubigeo <chr>, llave_distrito <chr>, dpto_hecho_new <chr>,
#   prov_hecho <chr>, dist_hecho <chr>, denuncias_2024_2026 <dbl>,
#   vehiculos_pnp <dbl>, vehiculos_operativos <dbl>, comisarias <int>
```

Esta tabla lista los registros que incumplen alguna regla de calidad. Si queda vacía, las reglas definidas no encontraron problemas.

11.3. Diagnóstico de fusiones

No basta con hacer un `left_join()`. También debemos revisar qué registros cruzaron y cuáles quedaron fuera.

```
if (!exists("base_distrital")) {
  preparar_fusiones()
}

parque_sin_denuncias <- parque_distrito |>
  # anti_join() devuelve registros del primer objeto que no cruzan con el segundo.
  anti_join(denuncias_distrito, by = "ubigeo")
```

```
denuncias_sin_parque <- denuncias_distrito |>
  anti_join(parque_distrito, by = "ubigeo")

comisarias_sin_denuncias <- comisarias_distrito |>
  anti_join(denuncias_distrito, by = "llave_distrito")

diagnostico_fusiones <- tibble(
  revision = c(
    "Distritos en parque automotor sin denuncias",
    "Distritos en denuncias sin parque automotor",
    "Distritos con comisariías sin denuncias"
  ),
  registros = c(
    nrow(parque_sin_denuncias),
    nrow(denuncias_sin_parque),
    nrow(comisarias_sin_denuncias)
  )
)

diagnostico_fusiones
```

```
# A tibble: 3 x 2
  revision                                registros
  <chr>                                <int>
1 Distritos en parque automotor sin denuncias      4
2 Distritos en denuncias sin parque automotor    799
3 Distritos con comisariías sin denuncias        99
```

El diagnóstico de fusiones muestra dónde se pierde correspondencia entre bases. Esta revisión es tan importante como el `left_join()`, porque revela límites del cruce.

```
denuncias_sin_parque |>
  arrange(desc(denuncias_2024_2026)) |>
  select(dpto_hecho_new, prov_hecho, dist_hecho, denuncias_2024_2026) |>
  head(15)
```

```
# A tibble: 15 x 4
  dpto_hecho_new      prov_hecho      dist_hecho      denuncias_2024_2026
  <chr>              <chr>          <chr>          <dbl>
1 UCAYALI            CORONEL PORTILLO MANANTAY      4276
2 REGION LIMA        HUARACHIRI     SAN ANTONIO    3026
```

3	AYACUCHO	HUAMANGA	SAN JUAN BAUTIS~	2230
4	REGION LIMA	HUAURA	SANTA MARIA	1714
5	PUNO	SAN ROMAN	SAN MIGUEL	1515
6	JUNIN	HUANCAYO	PILCOMAYO	1395
7	AYACUCHO	HUAMANGA	ANDRES AVELINO ~	1076
8	APURIMAC	ABANCAY	TAMBURCO	929
9	LAMBAYEQUE	FERREÑAFE	PUEBLO NUEVO	820
10	PROV. CONST. DEL CALLAO	CALLAO	MI PERU	798
11	LAMBAYEQUE	CHICLAYO	PUCALA	776
12	CUSCO	CUSCO	POROY	751
13	PASCO	OXAPAMPA	CONSTITUCION	702
14	CUSCO	QUISPICANCHI	OROPESA	593
15	JUNIN	CHUPACA	HUAMANCACA CHICO	576

Estos son distritos con denuncias, pero sin parque automotor cruzado por ubigeo. Puede deberse a ausencia real de registro o a diferencias en la codificación territorial.

11.4. Reestructurar datos: largo y ancho

Muchas bases se analizan mejor en formato largo: una fila por unidad y por indicador. Para reportes, a veces conviene volver a formato ancho.

```
if (!exists("base_distrital")) {
  preparar_fusiones()
}

base_indicadores_larga <- base_distrital |>
# El formato largo facilita comparar varios indicadores con una misma estructura.
select(
  ubigeo,
  dpto_hecho_new,
  prov_hecho,
  dist_hecho,
  denuncias_2024_2026,
  vehiculos_pnp,
  vehiculos_operativos,
  comisarias
) |>
pivot_longer(
  cols = c(
    denuncias_2024_2026,
```



```

    vehiculos_pnp,
    vehiculos_operativos,
    comisarias
  ),
  names_to = "indicador",
  values_to = "valor"
)

base_indicadores_larga

```

```

# A tibble: 7,232 x 6
  ubigeo dpto_hecho_new prov_hecho dist_hecho indicador      valor
  <chr>   <chr>          <chr>    <chr>    <chr>      <dbl>
1 010101 AMAZONAS      CHACHAPOYAS CHACHAPOYAS denuncias_2024_2026 3775
2 010101 AMAZONAS      CHACHAPOYAS CHACHAPOYAS vehiculos_pnp        65
3 010101 AMAZONAS      CHACHAPOYAS CHACHAPOYAS vehiculos_operativos  30
4 010101 AMAZONAS      CHACHAPOYAS CHACHAPOYAS comisarias           1
5 010102 AMAZONAS      CHACHAPOYAS ASUNCION    denuncias_2024_2026  14
6 010102 AMAZONAS      CHACHAPOYAS ASUNCION    vehiculos_pnp         0
7 010102 AMAZONAS      CHACHAPOYAS ASUNCION    vehiculos_operativos  0
8 010102 AMAZONAS      CHACHAPOYAS ASUNCION    comisarias           0
9 010103 AMAZONAS      CHACHAPOYAS BALSAS     denuncias_2024_2026  74
10 010103 AMAZONAS      CHACHAPOYAS BALSAS     vehiculos_pnp         0
# i 7,222 more rows

```

El formato largo es útil para comparar indicadores en un mismo campo (`indicador`) y facilita gráficos o resúmenes automatizados.

```

base_indicadores_larga |>
  group_by(indicador) |>
  summarise(
    minimo = min(valor, na.rm = TRUE),
    mediana = median(valor, na.rm = TRUE),
    promedio = mean(valor, na.rm = TRUE),
    maximo = max(valor, na.rm = TRUE),
    .groups = "drop"
  )

```

```

# A tibble: 4 x 5
  indicador      minimo mediana promedio maximo
  <chr>          <dbl>   <dbl>    <dbl>  <dbl>

```

1	comisarias	0	1	0.673	11
2	denuncias_2024_2026	1	77	1154.	74349
3	vehiculos_operativos	0	1	6.75	1726
4	vehiculos_pnp	0	1	11.3	2692

Este resumen permite detectar escalas muy distintas entre indicadores. Por ejemplo, denuncias, vehículos y comisarias no tienen magnitudes comparables sin estandarización.

```
base_indicadores_ancha <- base_indicadores_larga |>
# Volvemos al formato ancho cuando queremos una columna por indicador.
pivot_wider(
  names_from = indicador,
  values_from = valor
)

base_indicadores_ancha
```

```
# A tibble: 1,808 x 8
  ubigeo dpto_hecho_new prov_hecho dist_hecho denuncias_2024_2026 vehiculos_pnp
  <chr>   <chr>          <chr>   <chr>          <dbl>         <dbl>
1 010101 AMAZONAS      CHACHAPOY~ CHACHAPOY~      3775          65
2 010102 AMAZONAS      CHACHAPOY~ ASUNCION         14           0
3 010103 AMAZONAS      CHACHAPOY~ BALSAS          74           0
4 010104 AMAZONAS      CHACHAPOY~ CHETO           8           0
5 010105 AMAZONAS      CHACHAPOY~ CHILIQUN         8           0
6 010106 AMAZONAS      CHACHAPOY~ CHUQUIBAM~      33           0
7 010107 AMAZONAS      CHACHAPOY~ GRANADA          3           0
8 010108 AMAZONAS      CHACHAPOY~ HUANCAS         13           0
9 010109 AMAZONAS      CHACHAPOY~ LA JALCA       162           0
10 010110 AMAZONAS      CHACHAPOY~ LEIMEBAMBA     110           0
# i 1,798 more rows
# i 2 more variables: vehiculos_operativos <dbl>, comisarias <dbl>
```

El formato ancho es más cómodo para revisar una base final por distrito, mientras que el formato largo suele ser mejor para visualización y modelamiento.

12. Exportar una base limpia

Guardamos en esta misma carpeta la base distrital construida, su diccionario mínimo y la tabla en formato largo.

```

if (!exists("base_distrital")) {
  preparar_fusiones()
}

if (!exists("diccionario_base_distrital")) {
  diccionario_base_distrital <- tibble(
    variable = names(base_distrital),
    tipo = map_chr(base_distrital, ~ class(.x)[1]),
    descripcion = "Variable de la base distrital integrada."
  )
}

if (!exists("base_indicadores_larga")) {
  base_indicadores_larga <- base_distrital |>
    select(
      ubigeo,
      dpto_hecho_new,
      prov_hecho,
      dist_hecho,
      denuncias_2024_2026,
      vehiculos_pnp,
      vehiculos_operativos,
      comisarias
    ) |>
    pivot_longer(
      cols = c(
        denuncias_2024_2026,
        vehiculos_pnp,
        vehiculos_operativos,
        comisarias
      ),
      names_to = "indicador",
      values_to = "valor"
    )
}

write_csv(
  # Exportamos la base integrada principal.
  base_distrital,
  file.path(ruta_resultados, "base_sesion2.csv")
)

```

```

write_csv(
  # Exportamos el diccionario para documentar la base.
  diccionario_base_distrital,
  file.path(ruta_resultados, "diccionario_sesion2.csv")
)

write_csv(
  # Exportamos una versión en formato largo para ejercicios posteriores.
  base_indicadores_larga,
  file.path(ruta_resultados, "indicadores_sesion2.csv")
)

```

La exportación deja productos reutilizables: la base integrada, un diccionario mínimo y una versión larga para ejercicios posteriores.

13. Ejercicios para seguir practicando

1. Usemos `plotly` para explorar interactivamente la relación entre denuncias y comisarías por distrito.
2. Identificar la modalidad más frecuente en cada departamento.
3. Calcular el porcentaje de vehículos operativos por distrito.
4. Encontrar distritos con denuncias policiales, pero sin vehículos PNP registrados.
5. Revisar si hay distritos con `ubigeo` presente en parque automotor, pero no en denuncias.
6. Repetir el join usando solo Lima Metropolitana.
7. Crear una tabla de denuncias por modalidad y año.
8. Exportar una base resumida por departamento.

Estos ejercicios pueden apoyarse en el Posit Assistant de RStudio para revisar errores, pedir explicaciones de código y explorar alternativas de limpieza o visualización.

13.1. Ejercicio 1: Gráfico interactivo con `plotly`

Usamos `plotly` para crear un gráfico interactivo que muestre la relación entre denuncias y comisarías por distrito. Este tipo de gráfico permite explorar los datos de manera dinámica, mostrando información detallada al pasar el cursor sobre cada punto.

Sin embargo, no será visible en el PDF generado, ya que `plotly` está diseñado para entornos HTML. Para verlo, es necesario ejecutar el bloque de código en RStudio, Positron o renderizar el documento en formato HTML.

```

library(plotly)
if (!exists("base_distrital")) {
  preparar_fusiones()
}

grafico_comisarias <- base_distrital |>
  filter(comisarias > 0) |>
  ggplot(
    aes(
      x = comisarias,
      y = denuncias_2024_2026,
      text = paste0(
        "<b>Distrito:</b> ",
        dist_hecho,
        "<br><b>Departamento:</b> ",
        dpto_hecho_new,
        "<br><b>Comisarias:</b> ",
        comisarias,
        "<br><b>Denuncias:</b> ",
        scales::comma(denuncias_2024_2026)
      )
    )
  ) +
  geom_jitter(
    width = 0.12,
    height = 0,
    alpha = 0.45,
    color = "#2f4858",
    size = 2
  ) +
  scale_x_continuous(
    breaks = pretty_breaks(),
    expand = expansion(mult = c(0.08, 0.12))
  ) +
  scale_y_continuous(
    labels = comma,
    expand = expansion(mult = c(0.03, 0.08))
  ) +
  labs(
    title = "Denuncias y número de comisarias por distrito",
    x = "Comisarias",
    y = "Denuncias 2024-2026"
  )

```

```
) +  
  tema_sesion()  
  
ggplotly(  
  grafico_comisarias,  
  tooltip = "text"  
)
```